

[SoftwareBitmapSource](#) object and call it [SetBitmapAsync](#) to assign the software bitmap to the source. Finally, set the **Image** control's [Source](#) property to display the captured photo in the UI.

C#

```
SoftwareBitmap softwareBitmapBGR8 = SoftwareBitmap.Convert(softwareBitmap,
    BitmapPixelFormat.Bgra8,
    BitmapAlphaMode.Premultiplied);

SoftwareBitmapSource bitmapSource = new SoftwareBitmapSource();
await bitmapSource.SetBitmapAsync(softwareBitmapBGR8);

imageControl.Source = bitmapSource;
```

Capture a video with CameraCaptureUI

To capture a video, create a new [CameraCaptureUI](#) object. By using the object's [VideoSettings](#) property, you can specify properties for the returned video, such as the format of the video.

Call [CaptureFileAsync](#) and specify [Video](#) to capture a video. The method returns a [StorageFile](#) instance containing the video if the capture is successful. If you cancel the capture, the returned object is null.

C#

```
CameraCaptureUI captureUI = new CameraCaptureUI();
captureUI.VideoSettings.Format = CameraCaptureUIVideoFormat.Mp4;

StorageFile videoFile = await
captureUI.CaptureFileAsync(CameraCaptureUIMode.Video);

if (videoFile == null)
{
    // User cancelled photo capture
    return;
}
```

What you do with the captured video file depends on the scenario for your app. The rest of this article shows you how to quickly create a media composition from one or more captured videos and show it in your UI.

First, add a [MediaPlayerElement](#) control in which the video composition will display on your XAML page.

XML

```
<MediaPlayerElement x:Name="mediaPlayerElement" Width="320" Height="240"
AreTransportControlsEnabled="True"/>
```

When the video file returns from the camera capture UI, create a new [MediaSource](#) by calling [CreateFromStorageFile](#). Call the [Play](#) method of the default [MediaPlayer](#) associated with the **MediaPlayerElement** to play the video.

C#

```
mediaPlayerElement.Source = MediaSource.CreateFromStorageFile(videoFile);
mediaPlayerElement.MediaPlayer.Play();
```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)
- [CameraCaptureUI](#)

Display the camera preview

Article • 10/20/2022

This article describes how to quickly display the camera preview stream within a XAML page in a Universal Windows Platform (UWP) app. Creating an app that captures photos and videos using the camera requires you to perform tasks like handling device and camera orientation or setting encoding options for the captured file. For some app scenarios, you may want to just simply show the preview stream from the camera without worrying about these other considerations. This article shows you how to do that with a minimum of code. Note that you should always shut down the preview stream properly when you are done with it by following the steps below.

For information on writing a camera app that captures photos or videos, see [Basic photo, video, and audio capture with MediaCapture](#).

Add capability declarations to the app manifest

In order for your app to access a device's camera, you must declare that your app uses the *webcam* and *microphone* device capabilities.

Add capabilities to the app manifest

1. In Microsoft Visual Studio, in **Solution Explorer**, open the designer for the application manifest by double-clicking the **package.appxmanifest** item.
2. Select the **Capabilities** tab.
3. Check the box for **Webcam** and the box for **Microphone**.

Add a CaptureElement to your page

Use a [CaptureElement](#) to display the preview stream within your XAML page.

XML

```
<CaptureElement Name="PreviewControl" Stretch="Uniform"/>
```

Use MediaCapture to start the preview stream

The [MediaCapture](#) object is your app's interface to the device's camera. This class is a member of the `Windows.Media.Capture` namespace. The example in this article also

uses APIs from the [Windows.ApplicationModel](#) and [System.Threading.Tasks](#) namespaces, in addition to those included by the default project template.

Add using directives to include the following namespaces in your page's .cs file.

C#

```
using Windows.UI.Core;
using Windows.UI.Xaml.Navigation;
using Windows.Media.Capture;
using Windows.ApplicationModel;
using System.Threading.Tasks;
using Windows.System.Display;
using Windows.Graphics.Display;
```

Declare a class member variable for the **MediaCapture** object and a boolean to track whether the camera is currently previewing.

C#

```
MediaCapture mediaCapture;
bool isPreviewing;
```

Declare a variable of type [DisplayRequest](#) that will be used to make sure the display does not turn off while the preview is running.

C#

```
DisplayRequest displayRequest = new DisplayRequest();
```

Create a helper method to start up the camera preview, called **StartPreviewAsync** in this example. Depending on your app's scenario, you may want to call this from the **OnNavigatedTo** event handler that is called when the page is loaded or wait and launch the preview in response to UI events.

Create a new instance of the **MediaCapture** class and call [InitializeAsync](#) to initialize the capture device. This method may fail, on devices that don't have a camera for example, so you should call it from within a **try** block. An **UnauthorizedAccessException** will be thrown when you attempt to initialize the camera if the user has disabled camera access in the device's privacy settings. You will also see this exception during development if you have neglected to add the proper capabilities to your app manifest.

Important On some device families, a user consent prompt is displayed to the user before your app is granted access to the device's camera. For this reason, you must only

call [MediaCapture.InitializeAsync](#) from the main UI thread. Attempting to initialize the camera from another thread may result in initialization failure.

ⓘ Note

Windows allows users to grant or deny access to the device's camera in the Windows Settings app, under **Privacy & Security -> Camera**. When initializing the capture device, apps should check whether they have access to the camera and handle the case where access is denied by the user. For more information, see [Handle the Windows camera privacy setting](#).

Connect the **MediaCapture** to the **CaptureElement** by setting the [Source](#) property. Start the preview by calling [StartPreviewAsync](#). This method will throw a **FileLoadException** if another app has exclusive control of the capture device. See the next section for information listening for changes in exclusive control.

Call [RequestActive](#) to make sure the device doesn't go to sleep while the preview is running. Finally, set the [DisplayInformation.AutoRotationPreferences](#) property to [Landscape](#) to prevent the UI and the **CaptureElement** from rotating when the user changes the device orientation. For more information on handling device orientation changes, see [Handle device orientation with MediaCapture](#).

C#

```
private async Task StartPreviewAsync()
{
    try
    {
        mediaCapture = new MediaCapture();
        await mediaCapture.InitializeAsync();

        displayRequest.RequestActive();
        DisplayInformation.AutoRotationPreferences =
DisplayOrientations.Landscape;
    }
    catch (UnauthorizedAccessException)
    {
        // This will be thrown if the user denied access to the
camera in privacy settings
        ShowMessageToUser("The app was denied access to the camera");
        return;
    }

    try
    {
        PreviewControl.Source = mediaCapture;
    }
}
```

```

        await mediaCapture.StartPreviewAsync();
        isPreviewing = true;
    }
    catch (System.IO.FileLoadException)
    {
        mediaCapture.CaptureDeviceExclusiveControlStatusChanged +=
        _mediaCapture_CaptureDeviceExclusiveControlStatusChanged;
    }
}

```

Handle changes in exclusive control

As stated in the previous section, **StartPreviewAsync** will throw a **FileLoadException** if another app has exclusive control of the capture device. Starting with Windows 10, version 1703, you can register a handler for the [MediaCapture.CaptureDeviceExclusiveControlStatusChanged](#) event, which is raised whenever the exclusive control status of the device changes. In the handler for this event, check the [MediaCaptureDeviceExclusiveControlStatusChangedEventArgs.Status](#) property to see what the current status is. If the new status is **SharedReadOnlyAvailable**, then you know you can't currently start the preview and you may want to update your UI to alert the user. If the new status is **ExclusiveControlAvailable**, then you can try starting the camera preview again.

C#

```

private async void
_mediaCapture_CaptureDeviceExclusiveControlStatusChanged(MediaCapture
sender, MediaCaptureDeviceExclusiveControlStatusChangedEventArgs args)
{
    if (args.Status ==
MediaCaptureDeviceExclusiveControlStatus.SharedReadOnlyAvailable)
    {
        ShowMessageToUser("The camera preview can't be displayed because
another app has exclusive access");
    }
    else if (args.Status ==
MediaCaptureDeviceExclusiveControlStatus.ExclusiveControlAvailable &&
!isPreviewing)
    {
        await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, async () =>
        {
            await StartPreviewAsync();
        });
    }
}
}

```

Shut down the preview stream

When you are done using the preview stream, you should always shut down the stream and properly dispose of the associated resources to ensure that the camera is available to other apps on the device. The required steps for shutting down the preview stream are:

- If the camera is currently previewing, call [StopPreviewAsync](#) to stop the preview stream. An exception will be thrown if you call [StopPreviewAsync](#) while the preview is not running.
- Set the [Source](#) property of the [CaptureElement](#) to null. Use [CoreDispatcher.RunAsync](#) to make sure this call is executed on the UI thread.
- Call the [MediaCapture](#) object's [Dispose](#) method to release the object. Again, use [CoreDispatcher.RunAsync](#) to make sure this call is executed on the UI thread.
- Set the [MediaCapture](#) member variable to null.
- Call [RequestRelease](#) to allow the screen to turn off when inactive.

C#

```
private async Task CleanupCameraAsync()
{
    if (mediaCapture != null)
    {
        if (isPreviewing)
        {
            await mediaCapture.StopPreviewAsync();
        }

        await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
        {
            PreviewControl.Source = null;
            if (displayRequest != null)
            {
                displayRequest.RequestRelease();
            }

            mediaCapture.Dispose();
            mediaCapture = null;
        });
    }
}
```

You should shut down the preview stream when the user navigates away from your page by overriding the [OnNavigatedFrom](#) method.

C#

```
protected async override void OnNavigatedFrom(NavigationEventArgs e)
{
    await CleanupCameraAsync();
}
```

You should also shut down the preview stream properly when your app is suspending. To do this, register a handler for the [Application.Suspending](#) event in your page's constructor.

```
C#

public MainPage()
{
    this.InitializeComponent();

    Application.Current.Suspending += Application_Suspending;
}
```

In the **Suspending** event handler, first check to make sure that the page is being displayed the application's **Frame** by comparing the page type to the **CurrentSourcePageType** property. If the page is not currently being displayed, then the **OnNavigatedFrom** event should already have been raised and the preview stream shut down. If the page is currently being displayed, get a **SuspendingDeferral** object from the event args passed into the handler to make sure the system does not suspend your app until the preview stream has been shut down. After shutting down the stream, call the deferral's **Complete** method to let the system continue suspending your app.

```
C#

private async void Application_Suspending(object sender, SuspendingEventArgs e)
{
    // Handle global application events only if this page is active
    if (Frame.CurrentSourcePageType == typeof(MainPage))
    {
        var deferral = e.SuspendingOperation.GetDeferral();
        await CleanupCameraAsync();
        deferral.Complete();
    }
}
```

Related topics

- [Camera](#)

- [Basic photo, video, and audio capture with MediaCapture](#)
- [Get a preview frame](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Basic photo, video, and audio capture with MediaCapture

Article • 10/20/2022

This article shows the simplest way to capture photos and video using the [MediaCapture](#) class. The **MediaCapture** class exposes a robust set of APIs that provide low-level control over the capture pipeline and enable advanced capture scenarios, but this article is intended to help you add basic media capture to your app quickly and easily. To learn about more of the features that **MediaCapture** provides, see [Camera](#).

If you simply want to capture a photo or video and don't intend to add any additional media capture features, or if you don't want to create your own camera UI, you may want to use the [CameraCaptureUI](#) class, which allows you to simply launch the Windows built-in camera app and receive the photo or video file that was captured. For more information, see [Capture photos and video with Windows built-in camera UI](#)

The code in this article was adapted from the [Camera starter kit](#) [↗](#) sample. You can download the sample to see the code used in context or to use the sample as a starting point for your own app.

Add capability declarations to the app manifest

In order for your app to access a device's camera, you must declare that your app uses the *webcam* and *microphone* device capabilities. If you want to save captured photos and videos to the users's Pictures or Videos library, you must also declare the *picturesLibrary* and *videosLibrary* capability.

To add capabilities to the app manifest

1. In Microsoft Visual Studio, in **Solution Explorer**, open the designer for the application manifest by double-clicking the **package.appxmanifest** item.
2. Select the **Capabilities** tab.
3. Check the box for **Webcam** and the box for **Microphone**.
4. For access to the Pictures and Videos library check the boxes for **Pictures Library** and the box for **Videos Library**.

Initialize the MediaCapture object

All of the capture methods described in this article require the first step of initializing the **MediaCapture** object by calling the constructor and then calling [InitializeAsync](#).

Since the **MediaCapture** object will be accessed from multiple places in your app, declare a class variable to hold the object. Implement a handler for the **MediaCapture** object's **Failed** event to be notified if a capture operation fails.

C#

```
MediaCapture mediaCapture;  
bool isPreviewing;
```

C#

```
mediaCapture = new MediaCapture();  
await mediaCapture.InitializeAsync();  
mediaCapture.Failed += MediaCapture_Failed;
```

ⓘ Note

Windows allows users to grant or deny access to the device's camera in the Windows Settings app, under **Privacy & Security** -> **Camera**. When initializing the capture device, apps should check whether they have access to the camera and handle the case where access is denied by the user. For more information, see [Handle the Windows camera privacy setting](#).

Set up the camera preview

It's possible to capture photos, videos, and audio using **MediaCapture** without showing the camera preview, but typically you want to show the preview stream so that the user can see what's being captured. Also, a few **MediaCapture** features require the preview stream to be running before they can be enabled, including auto focus, auto exposure, and auto white balance. To see how to set up the camera preview, see [Display the camera preview](#).

Capture a photo to a SoftwareBitmap

The [SoftwareBitmap](#) class was introduced in Windows 10 to provide a common representation of images across multiple features. If you want to capture a photo and then immediately use the captured image in your app, such as displaying it in XAML, instead of capturing to a file, then you should capture to a **SoftwareBitmap**. You still have the option of saving the image to disk later.

After initializing the **MediaCapture** object, you can capture a photo to a **SoftwareBitmap** using the **LowLagPhotoCapture** class. Get an instance of this class by calling **PrepareLowLagPhotoCaptureAsync**, passing in an **ImageEncodingProperties** object specifying the image format you want. **CreateUncompressed** creates an uncompressed encoding with the specified pixel format. Capture a photo by calling **CaptureAsync**, which returns a **CapturedPhoto** object. Get a **SoftwareBitmap** by accessing the **Frame** property and then the **SoftwareBitmap** property.

If you want, you can capture multiple photos by repeatedly calling **CaptureAsync**. When you are done capturing, call **FinishAsync** to shut down the **LowLagPhotoCapture** session and free up the associated resources. After calling **FinishAsync**, to begin capturing photos again you will need to call **PrepareLowLagPhotoCaptureAsync** again to reinitialize the capture session before calling **CaptureAsync**.

C#

```
// Prepare and capture photo
var lowLagCapture = await
mediaCapture.PrepareLowLagPhotoCaptureAsync(ImageEncodingProperties.CreateUn
compressed(MediaPixelFormat.Bgra8));

var capturedPhoto = await lowLagCapture.CaptureAsync();
var softwareBitmap = capturedPhoto.Frame.SoftwareBitmap;

await lowLagCapture.FinishAsync();
```

Starting with Windows, version 1803, you can access the **BitmapProperties** property of the **CapturedFrame** class returned from **CaptureAsync** to retrieve metadata about the captured photo. You can pass this data into a **BitmapEncoder** to save the metadata to a file. Previously, there was no way to access this data for uncompressed image formats. You can also access the **ControlValues** property to retrieve a **CapturedFrameControlValues** object that describes the control values, such as exposure and white balance, for the captured frame.

For information about using **BitmapEncoder** and about working with the **SoftwareBitmap** object, including how to display one in a XAML page, see [Create, edit, and save bitmap images](#).

For more information on setting capture device control values, see [Capture device controls for photo and video](#).

Starting with Windows 10, version 1803, you can get the metadata, such as EXIF information, for photos captured in uncompressed format by accessing the **BitmapProperties** property of the **CapturedFrame** returned by **MediaCapture**. In

previous releases this data was only accessible in the header of photos captured to a compressed file format. You can provide this data to a [BitmapEncoder](#) when manually writing an image file. For more information on encoding bitmaps, see [Create, edit, and save bitmap images](#). You can also access the frame control values, such as exposure and flash settings, used when the image was captured by accessing the [ControlValues](#) property. For more information, see [Capture device controls for photo and video capture](#).

Capture a photo to a file

A typical photography app will save a captured photo to disk or to cloud storage and will need to add metadata, such as photo orientation, to the file. The following example shows you how to capture a photo to a file. You still have the option of creating a [SoftwareBitmap](#) from the image file later.

The technique shown in this example captures the photo to an in-memory stream and then transcode the photo from the stream to a file on disk. This example uses [GetLibraryAsync](#) to get the user's pictures library and then the [SaveFolder](#) property to get a reference default save folder. Remember to add the **Pictures Library** capability to your app manifest to access this folder. [CreateFileAsync](#) creates a new [StorageFile](#) to which the photo will be saved.

Create an [InMemoryRandomAccessStream](#) and then call [CapturePhotoToStreamAsync](#) to capture a photo to the stream, passing in the stream and an [ImageEncodingProperties](#) object specifying the image format that should be used. You can create custom encoding properties by initializing the object yourself, but the class provides static methods, like [ImageEncodingProperties.CreateJpeg](#) for common encoding formats. Next, create a file stream to the output file by calling [OpenAsync](#). Create a [BitmapDecoder](#) to decode the image from the in memory stream and then create a [BitmapEncoder](#) to encode the image to file by calling [CreateForTranscodingAsync](#).

You can optionally create a [BitmapPropertySet](#) object and then call [SetPropertiesAsync](#) on the image encoder to include metadata about the photo in the image file. For more information about encoding properties, see [Image metadata](#). Handling device orientation properly is essential for most photography apps. For more information, see [Handle device orientation with MediaCapture](#).

Finally, call [FlushAsync](#) on the encoder object to transcode the photo from the in-memory stream to the file.

```

var myPictures = await
Windows.Storage.StorageLibrary.GetLibraryAsync(Windows.Storage.KnownLibraryI
d.Pictures);
StorageFile file = await myPictures.SaveFolder.CreateFileAsync("photo.jpg",
CreationCollisionOption.GenerateUniqueName);

using (var captureStream = new InMemoryRandomAccessStream())
{
    await
mediaCapture.CapturePhotoToStreamAsync(ImageEncodingProperties.CreateJpeg(),
captureStream);

    using (var fileStream = await file.OpenAsync(FileAccessMode.ReadWrite))
    {
        var decoder = await BitmapDecoder.CreateAsync(captureStream);
        var encoder = await
BitmapEncoder.CreateForTranscodingAsync(fileStream, decoder);

        var properties = new BitmapPropertySet {
            { "System.Photo.Orientation", new
BitmapTypedValue(PhotoOrientation.Normal, PropertyType.UInt16) }
        };
        await encoder.BitmapProperties.SetPropertiesAsync(properties);

        await encoder.FlushAsync();
    }
}

```

For more information about working with files and folders, see [Files, folders, and libraries](#).

Capture a video

Quickly add video capture to your app by using the [LowLagMediaRecording](#) class. First, declare a class variable to for the object.

C#

```
LowLagMediaRecording _mediaRecording;
```

Next, create a **StorageFile** object to which the video will be saved. Note that to save to the user's video library, as shown in this example, you must add the **Videos Library** capability to your app manifest. Call [PrepareLowLagRecordToStorageFileAsync](#) to initialize the media recording, passing in the storage file and a [MediaEncodingProfile](#) object specifying the encoding for the video. The class provides static methods, like [CreateMp4](#), for creating common video encoding profiles.

Finally, call **StartAsync** to begin capturing video.

C#

```
var myVideos = await
Windows.Storage.StorageLibrary.GetLibraryAsync(Windows.Storage.KnownLibraryI
d.Videos);
StorageFile file = await myVideos.SaveFolder.CreateFileAsync("video.mp4",
CreationCollisionOption.GenerateUniqueName);
_mediaRecording = await mediaCapture.PrepareLowLagRecordToStorageFileAsync(
    MediaEncodingProfile.CreateMp4(VideoEncodingQuality.Auto), file);
await _mediaRecording.StartAsync();
```

To stop recording video, call **StopAsync**.

C#

```
await _mediaRecording.StopAsync();
```

You can continue to call **StartAsync** and **StopAsync** to capture additional videos. When you are done capturing videos, call **FinishAsync** to dispose of the capture session and clean up associated resources. After this call, you must call **PrepareLowLagRecordToStorageFileAsync** again to reinitialize the capture session before calling **StartAsync**.

C#

```
await _mediaRecording.FinishAsync();
```

When capturing video, you should register a handler for the **RecordLimitationExceeded** event of the **MediaCapture** object, which will be raised by the operating system if you surpass the limit for a single recording, currently three hours. In the handler for the event, you should finalize your recording by calling **StopAsync**.

C#

```
mediaCapture.RecordLimitationExceeded +=
MediaCapture_RecordLimitationExceeded;
```

C#

```
private async void MediaCapture_RecordLimitationExceeded(MediaCapture
sender)
{
    await _mediaRecording.StopAsync();
}
```

```
System.Diagnostics.Debug.WriteLine("Record limitation exceeded.");  
}
```

Play and edit captured video files

Once you have captured a video to a file, you may want to load the file and play it back within your app's UI. You can do this using the [MediaPlayerElement](#) XAML control and an associated [MediaPlayer](#). For information on playing media in a XAML page, see [Play audio and video with MediaPlayer](#).

You can also create a [MediaClip](#) object from a video file by calling [CreateFromFileAsync](#). A [MediaComposition](#) provides basic video editing functionality like arranging the sequence of [MediaClip](#) objects, trimming video length, creating layers, adding background music, and applying video effects. For more information on working with media compositions, see [Media compositions and editing](#).

Pause and resume video recording

You can pause a video recording and then resume recording without creating a separate output file by calling [PauseAsync](#) and then calling [ResumeAsync](#).

C#

```
await  
_mediaRecording.PauseAsync(Windows.Media.Devices.MediaCapturePauseBehavior.ReleaseHardwareResources);
```

C#

```
await _mediaRecording.ResumeAsync();
```

Starting with Windows 10, version 1607, you can pause a video recording and receive the last frame captured before the recording was paused. You can then overlay this frame on the camera preview to allow the user to align the camera with the paused frame before resuming recording. Calling [PauseWithResultAsync](#) returns a [MediaCapturePauseResult](#) object. The [LastFrame](#) property is a [VideoFrame](#) object representing the last frame. To display the frame in XAML, get the [SoftwareBitmap](#) representation of the video frame. Currently, only images in BGRA8 format with premultiplied or empty alpha channel are supported, so call [Convert](#) if necessary to get the correct format. Create a new [SoftwareBitmapSource](#) object and call [SetBitmapAsync](#) to initialize it. Finally, set the [Source](#) property of a XAML [Image](#) control

to display the image. For this trick to work, your image must be aligned with the **CaptureElement** control and should have an opacity value less than one. Don't forget that you can only modify the UI on the UI thread, so make this call inside [RunAsync](#).

PauseWithResultAsync also returns the duration of the video that was recorded in the preceding segment in case you need to track how much total time has been recorded.

C#

```
MediaCapturePauseResult result =
    await
_mediaRecording.PauseWithResultAsync(Windows.Media.Devices.MediaCapturePause
Behavior.RetainHardwareResources);

var pausedFrame = result.LastFrame.SoftwareBitmap;
if(pausedFrame.BitmapPixelFormat != BitmapPixelFormat.Bgra8 ||
pausedFrame.BitmapAlphaMode != BitmapAlphaMode.Ignore)
{
    pausedFrame = SoftwareBitmap.Convert(pausedFrame,
BitmapPixelFormat.Bgra8, BitmapAlphaMode.Ignore);
}

var source = new SoftwareBitmapSource();
await source.SetBitmapAsync(pausedFrame);

await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
{
    PauseImage.Source = source;
    PauseImage.Visibility = Visibility.Visible;
});

_totalRecordedTime += result.RecordDuration;
```

When you resume recording, you can set the source of the image to null and hide it.

C#

```
await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
{
    PauseImage.Source = null;
    PauseImage.Visibility = Visibility.Collapsed;
});

await _mediaRecording.ResumeAsync();
```

Note that you can also get a result frame when you stop the video by calling [StopWithResultAsync](#).

Capture audio

You can quickly add audio capture to your app by using the same technique shown above for capturing video. The example below creates a **StorageFile** in the application data folder. Call **PrepareLowLagRecordToStorageFileAsync** to initialize the capture session, passing in the file and a **MediaEncodingProfile** which is generated in this example by the **CreateMp3** static method. To begin recording, call **StartAsync**.

C#

```
mediaCapture.RecordLimitationExceeded +=
MediaCapture_RecordLimitationExceeded;

var localFolder = Windows.Storage.ApplicationData.Current.LocalFolder;
StorageFile file = await localFolder.CreateFileAsync("audio.mp3",
CreationCollisionOption.GenerateUniqueName);
_mediaRecording = await mediaCapture.PrepareLowLagRecordToStorageFileAsync(
    MediaEncodingProfile.CreateMp3(AudioEncodingQuality.High), file);
await _mediaRecording.StartAsync();
```

Call **StopAsync** to stop the audio recording.

Related topics

- [Camera](#)

C#

```
await _mediaRecording.StopAsync();
```

You can call **StartAsync** and **StopAsync** multiple times to record several audio files. When you are done capturing audio, call **FinishAsync** to dispose of the capture session and clean up associated resources. After this call, you must call **PrepareLowLagRecordToStorageFileAsync** again to reinitialize the capture session before calling **StartAsync**.

C#

```
await _mediaRecording.FinishAsync();
```

Detect and respond to audio level changes by the system

Starting with Windows 10, version 1803, your app can detect when the system lowers or mutes the audio level of your app's audio capture and audio render streams. For example, the system may mute your app's streams when it goes into the background. The [AudioStateMonitor](#) class allows you to register to receive an event when the system modifies the volume of an audio stream. Get an instance of **AudioStateMonitor** for monitoring audio capture streams by calling [CreateForCaptureMonitoring](#). Get an instance for monitoring audio render streams by calling [CreateForRenderMonitoring](#). Register a handler for the [SoundLevelChanged](#) event of each monitor to be notified when the audio for the corresponding stream category is changed by the system.

C#

```
// Namespaces for monitoring audio state
using Windows.Media;
using Windows.Media.Audio;
```

C#

```
AudioStateMonitor captureAudioStateMonitor;
AudioStateMonitor renderAudioStateMonitor;
```

C#

```
captureAudioStateMonitor = AudioStateMonitor.CreateForCaptureMonitoring();
captureAudioStateMonitor.SoundLevelChanged +=
    CaptureAudioStateMonitor_SoundLevelChanged; ;

renderAudioStateMonitor = AudioStateMonitor.CreateForRenderMonitoring();
renderAudioStateMonitor.SoundLevelChanged +=
    RenderAudioStateMonitor_SoundLevelChanged; ;
```

In the **SoundLevelChanged** handler for the capture stream, you can check the [SoundLevel](#) property of the **AudioStateMonitor** sender to determine the new sound level. Note that a capture stream should never be lowered, or "ducked", by the system. It should only ever be muted or switched back to full volume. If the audio stream is muted, you can stop a capture in progress. If the audio stream is restored to full volume, you can start capturing again. The following example uses some boolean class variables to track whether the app is currently capturing audio and if the capture was stopped due to the audio state change. These variables are used to determine when it's appropriate to programmatically stop or start audio capture.

C#

```

bool isCapturingAudio = false;
bool capturingStoppedForAudioState = false;
private void CaptureAudioStateMonitor_SoundLevelChanged(AudioStateMonitor
sender, object args)
{
    switch (sender.SoundLevel)
    {
        case SoundLevel.Full:
            if(capturingStoppedForAudioState)
            {
                StartAudioCapture();
                capturingStoppedForAudioState = false;
            }
            break;
        case SoundLevel.Muted:
            if(isCapturingAudio)
            {
                StopAudioCapture();
                capturingStoppedForAudioState = true;
            }
            break;
        case SoundLevel.Low:
            // This should never happen for capture
            Debug.WriteLine("Unexpected audio state.");
            break;
    }
}

```

The following code example illustrates an implementation of the **SoundLevelChanged** handler for audio rendering. Depending on your app scenario, and the type of content you are playing, you may want to pause audio playback when the sound level is ducked. For more information on handling sound level changes for media playback, see [Play audio and video with MediaPlayer](#).

C#

```

private void RenderAudioStateMonitor_SoundLevelChanged(AudioStateMonitor
sender, object args)
{
    if ((sender.SoundLevel == SoundLevel.Full) ||
(sender.SoundLevel == SoundLevel.Low && !isPodcast))
    {
        mediaPlayer.Play();
    }
    else if ((sender.SoundLevel == SoundLevel.Muted) ||
(sender.SoundLevel == SoundLevel.Low && isPodcast))
    {
        // Pause playback if we're muted or if we're playing a podcast and
are ducked
        mediaPlayer.Pause();
    }
}

```

```
}  
}
```

- [Capture photos and video with Windows built-in camera UI](#)
- [Handle device orientation with MediaCapture](#)
- [Create, edit, and save bitmap images](#)
- [Files, folders, and libraries](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Camera UI features for mobile devices

Article • 10/20/2022

This article show you how to take advantage of special camera UI features that are only present on mobile devices.

Add the mobile extension to your project

To use these features, you must add a reference to the Microsoft Mobile Extension SDK for Universal App Platform to your project.

To add a reference to the mobile extension SDK for hardware camera button support

1. In **Solution Explorer**, right-click **References** and select **Add Reference**.
2. Expand the **Windows Universal** node and select **Extensions**.
3. Select the **Microsoft Mobile Extension SDK for Universal App Platform** check box.

Hide the status bar

Mobile devices have a [StatusBar](#) control that provides the user with status information about the device. This control takes up space on the screen that can interfere with the media capture UI. You can hide the status bar by calling [HideAsync](#), but you must make this call from within a conditional block where you use the [ApiInformation.IsTypePresent](#) method to determine if the API is available. This method will only return true on mobile devices that support the status bar. You should hide the status bar when your app launches or when you begin previewing from the camera.

C#

```
// Hide the status bar
if (ApiInformation.IsTypePresent("Windows.UI.ViewManagement.StatusBar"))
{
    await
    Windows.UI.ViewManagement.StatusBar.GetForCurrentView().HideAsync();
}
```

When your app is shutting down or when the user navigates away from the media capture page of your app, you can make the control visible again.

C#

```
// Show the status bar
if (ApiInformation.IsTypePresent("Windows.UI.ViewManagement.StatusBar"))
{
    await
Windows.UI.ViewManagement.StatusBar.GetForCurrentView().ShowAsync();
}
```

Use the hardware camera button

Some mobile devices have a dedicated hardware camera button that some users prefer over an on-screen control. To be notified when the hardware camera button is pressed, register a handler for the [HardwareButtons.CameraPressed](#) event. Because this API is available on mobile devices only, you must again use the **IsTypePresent** to make sure the API is supported on the current device before attempting to access it.

C#

```
using Windows.Phone.UI.Input;
```

C#

```
if (ApiInformation.IsTypePresent("Windows.Phone.UI.Input.HardwareButtons"))
{
    HardwareButtons.CameraPressed += HardwareButtons_CameraPressed;
}
```

In the handler for the **CameraPressed** event, you can initiate a photo capture.

C#

```
private async void HardwareButtons_CameraPressed(object sender,
CameraEventArgs e)
{
    await TakePhotoAsync();
}
```

When your app is shutting down or the user moves away from the media capture page of your app, unregister the hardware button handler.

C#

```
if (ApiInformation.IsTypePresent("Windows.Phone.UI.Input.HardwareButtons"))
{
    HardwareButtons.CameraPressed -= HardwareButtons_CameraPressed;
}
```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)

Handle device orientation with MediaCapture

Article • 10/20/2022

When your app captures a photo or video that is intended to be viewed outside of your app, such as saving to a file on the user's device or sharing online, it's important that you encode the image with the proper orientation metadata so that when another app or device displays the image, it is oriented correctly. Determining the correct orientation data to include in a media file can be a complex task because there are several variables to consider, including the orientation of the device chassis, the orientation of the display, and the placement of the camera on the chassis (whether it is a front or back-facing camera).

To simplify the process of handling orientation, we recommend using a helper class, **CameraRotationHelper**, for which the full definition is provided at the end of this article. You can add this class to your project and then follow the steps in this article to add orientation support to your camera app. The helper class also makes it easier for you to rotate the controls in your camera UI so that they are rendered correctly from the user's point of view.

ⓘ Note

This article builds on the code and concepts discussed in the article [Basic photo, video, and audio capture with MediaCapture](#). We recommend that you familiarize yourself with the basic concepts of using the **MediaCapture** class before adding orientation support to your app.

Namespaces used in this article

The example code in this article uses APIs from the following namespaces that you should include in your code.

C#

```
using Windows.Devices.Enumeration;  
using Windows.UI.Core;
```

The first step in adding orientation support to your app is to lock the display so that it doesn't automatically rotate when the device is rotated. Automatic UI rotation works

well for most types of apps, but it is unintuitive for users when the camera preview rotates. Lock the display orientation by setting the [DisplayInformation.AutoRotationPreferences](#) property to [DisplayOrientations.Landscape](#).

C#

```
DisplayInformation.AutoRotationPreferences = DisplayOrientations.Landscape;
```

Tracking the camera device location

To calculate the correct orientation for captured media, your app must determine the location of the camera device on the chassis. Add a boolean member variable to track whether the camera is external to the device, such as a USB web cam. Add another boolean variable to track whether the preview should be mirrored, which is the case if a front-facing camera is used. Also, add a variable for storing a **DeviceInformation** object that represents the selected camera.

C#

```
private bool _externalCamera;  
private bool _mirroringPreview;
```

C#

```
DeviceInformation _cameraDevice;
```

Select a camera device and initialize the MediaCapture object

The article [Basic photo, video, and audio capture with MediaCapture](#) shows you how to initialize the **MediaCapture** object with just a couple of lines of code. To support camera orientation, we will add a few more steps to the initialization process.

First, call [DeviceInformation.FindAllAsync](#) passing in the device selector [DeviceClass.VideoCapture](#) to get a list of all available video capture devices. Next, select the first device in the list where the panel location of the camera is known and where it matches the supplied value, which in this example is a front-facing camera. If no camera is found on the desired panel, the first or default available camera is used.

If a camera device is found, a new [MediaCaptureInitializationSettings](#) object is created and the [VideoDeviceId](#) property is set to the selected device. Next, create the MediaCapture object and call [InitializeAsync](#), passing in the settings object to tell the system to use the selected camera.

Finally, check to see if the selected device panel is null or unknown. If so, the camera is external, which means that its rotation is unrelated to the rotation of the device. If the panel is known and is on the front of the device chassis, we know the preview should be mirrored, so the variable tracking this is set.

C#

```
var allVideoDevices = await
DeviceInformation.FindAllAsync(DeviceClass.VideoCapture);
DeviceInformation desiredDevice = allVideoDevices.FirstOrDefault(x =>
x.EnclosureLocation != null
    && x.EnclosureLocation.Panel ==
Windows.Devices.Enumeration.Panel.Front);
_cameraDevice = desiredDevice ?? allVideoDevices.FirstOrDefault();

if (_cameraDevice == null)
{
    System.Diagnostics.Debug.WriteLine("No camera device found!");
    return;
}

var settings = new MediaCaptureInitializationSettings { VideoDeviceId =
_cameraDevice.Id };

mediaCapture = new MediaCapture();
mediaCapture.RecordLimitationExceeded +=
MediaCapture_RecordLimitationExceeded;
mediaCapture.Failed += MediaCapture_Failed;

try
{
    await mediaCapture.InitializeAsync(settings);
}
catch (UnauthorizedAccessException)
{
    System.Diagnostics.Debug.WriteLine("The app was denied access to the
camera");
    return;
}

// Handle camera device location
if (_cameraDevice.EnclosureLocation == null ||
    _cameraDevice.EnclosureLocation.Panel ==
Windows.Devices.Enumeration.Panel.Unknown)
{
    _externalCamera = true;
```

```

}
else
{
    _externalCamera = false;
    _mirroringPreview = (_cameraDevice.EnclosureLocation.Panel ==
Windows.Devices.Enumeration.Panel.Front);
}

```

Initialize the CameraRotationHelper class

Now we begin using the **CameraRotationHelper** class. Declare a class member variable to store the object. Call the constructor, passing in the enclosure location of the selected camera. The helper class uses this information to calculate the correct orientation for captured media, the preview stream, and the UI. Register a handler for the helper class's **OrientationChanged** event, which will be raised when we need to update the orientation of the UI or the preview stream.

C#

```
private CameraRotationHelper _rotationHelper;
```

C#

```
_rotationHelper = new CameraRotationHelper(_cameraDevice.EnclosureLocation);
_rotationHelper.OrientationChanged += RotationHelper_OrientationChanged;
```

Add orientation data to the camera preview stream

Adding the correct orientation to the metadata of the preview stream does not affect how the preview appears to the user, but it helps the system encode any frames captured from the preview stream correctly.

You start the camera preview by calling [MediaCapture.StartPreviewAsync](#). Before you do this, check the member variable to see if the preview should be mirrored (for a front-facing camera). If so, set the [FlowDirection](#) property of the [CaptureElement](#), named *PreviewControl* in this example, to [FlowDirection.RightToLeft](#). After starting the preview, call the helper method **SetPreviewRotationAsync** to set the preview rotation. Following is the implementation of this method.

C#

```

PreviewControl.Source = mediaCapture;
PreviewControl.FlowDirection = _mirroringPreview ? FlowDirection.RightToLeft
: FlowDirection.LeftToRight;

await mediaCapture.StartPreviewAsync();
await SetPreviewRotationAsync();

```

We set the preview rotation in a separate method so that it can be updated when the phone orientation changes without reinitializing the preview stream. If the camera is external to the device, no action is taken. Otherwise, the **CameraRotationHelper** method **GetCameraPreviewOrientation** is called and returns the proper orientation for the preview stream.

To set the metadata, the preview stream properties are retrieved by calling **VideoDeviceController.GetMediaStreamProperties**. Next, create the GUID representing the Media Foundation Transform (MFT) attribute for video stream rotation. In C++ you can use the constant **MF_MT_VIDEO_ROTATION**, but in C# you must manually specify the GUID value.

Add a property value to the stream properties object, specifying the GUID as the key and the preview rotation as the value. This property expects values to be in units of counterclockwise degrees, so the **CameraRotationHelper** method **ConvertSimpleOrientationToClockwiseDegrees** is used to convert the simple orientation value. Finally, call **SetEncodingPropertiesAsync** to apply the new rotation property to the stream.

C#

```

private async Task SetPreviewRotationAsync()
{
    if (!_externalCamera)
    {
        // Add rotation metadata to the preview stream to make sure the
        // aspect ratio / dimensions match when rendering and getting preview frames
        var rotation = _rotationHelper.GetCameraPreviewOrientation();
        var props =
mediaCapture.VideoDeviceController.GetMediaStreamProperties(MediaStreamType.
VideoPreview);
        Guid RotationKey = new Guid("C380465D-2271-428C-9B83-ECEA3B4A85C1");
        props.Properties.Add(RotationKey,
CameraRotationHelper.ConvertSimpleOrientationToClockwiseDegrees(rotation));
        await
mediaCapture.SetEncodingPropertiesAsync(MediaStreamType.VideoPreview, props,
null);
    }
}

```

Next, add the handler for the **CameraRotationHelper.OrientationChanged** event. This event passes in an argument that lets you know whether the preview stream needs to be rotated. If the orientation of the device was changed to face up or face down, this value will be false. If the preview does need to be rotated, call **SetPreviewRotationAsync** which was defined previously.

Next, in the **OrientationChanged** event handler, update your UI if needed. Get the current recommended UI orientation from the helper class by calling **GetUIOrientation** and convert the value to clockwise degrees, which is used for XAML transforms. Create a **RotateTransform** from the orientation value and set the **RenderTransform** property of your XAML controls. Depending on your UI layout, you may need to make additional adjustments here in addition to simply rotating the controls. Also, remember that all updates to your UI must be made on the UI thread, so you should place this code inside a call to **RunAsync**.

C#

```
private async void RotationHelper_OrientationChanged(object sender, bool
updatePreview)
{
    if (updatePreview)
    {
        await SetPreviewRotationAsync();
    }
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () => {
        // Rotate the buttons in the UI to match the rotation of the device
        var angle =
CameraRotationHelper.ConvertSimpleOrientationToClockwiseDegrees(_rotationHel
per.GetUIOrientation());
        var transform = new RotateTransform { Angle = angle };

        // The RenderTransform is safe to use (i.e. it won't cause layout
issues) in this case, because these buttons have a 1:1 aspect ratio
        CapturePhotoButton.RenderTransform = transform;
        CapturePhotoButton.RenderTransform = transform;
    });
}
```

Capture a photo with orientation data

The article [Basic photo, video, and audio capture with MediaCapture](#) shows you how to capture a photo to a file by capturing first to an in-memory stream and then using a decoder to read the image data from the stream and an encoder to transcode the image data to a file. Orientation data, obtained from the **CameraRotationHelper** class, can be added to the image file during the transcoding operation.

In the following example, a photo is captured to an [InMemoryRandomAccessStream](#) with a call to [CapturePhotoToStreamAsync](#) and a [BitmapDecoder](#) is created from the stream. Next a [StorageFile](#) is created and opened to retrieve an [IRandomAccessStream](#) for writing to the file.

Before transcoding the file, the photo's orientation is retrieved from the helper class method [GetCameraCaptureOrientation](#). This method returns a [SimpleOrientation](#) object which is converted to a [PhotoOrientation](#) object with the helper method [ConvertSimpleOrientationToPhotoOrientation](#). Next, a new [BitmapPropertySet](#) object is created and a property is added where the key is "System.Photo.Orientation" and the value is the photo orientation, expressed as a [BitmapTypedValue](#).

"System.Photo.Orientation" is one of many Windows properties that can be added as metadata to an image file. For a list of all of the photo-related properties, see [Windows Properties - Photo](#). For more information about working with metadata in images, see [Image metadata](#).

Finally, the property set which includes the orientation data is set for the encoder by with a call to [SetPropertiesAsync](#) and the image is transcoded with a call to [FlushAsync](#).

C#

```
private async Task CapturePhotoWithOrientationAsync()
{
    var captureStream = new InMemoryRandomAccessStream();

    try
    {
        await
mediaCapture.CapturePhotoToStreamAsync(ImageEncodingProperties.CreateJpeg(),
captureStream);
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine("Exception when taking a photo:
{0}", ex.ToString());
        return;
    }

    var decoder = await BitmapDecoder.CreateAsync(captureStream);
    var file = await
KnownFolders.PicturesLibrary.CreateFileAsync("SimplePhoto.jpeg",
CreationCollisionOption.GenerateUniqueName);

    using (var outputStream = await
file.OpenAsync(FileAccessMode.ReadWrite))
    {
        var encoder = await
BitmapEncoder.CreateForTranscodingAsync(outputStream, decoder);
```

```

        var photoOrientation =
        CameraRotationHelper.ConvertSimpleOrientationToPhotoOrientation(
            _rotationHelper.GetCameraCaptureOrientation());
        var properties = new BitmapPropertySet {
            { "System.Photo.Orientation", new
            BitmapTypedValue(photoOrientation, PropertyType.UInt16) } };
        await encoder.BitmapProperties.SetPropertiesAsync(properties);
        await encoder.FlushAsync();
    }
}

```

Capture a video with orientation data

Basic video capture is described in the article [Basic photo, video, and audio capture with MediaCapture](#). Adding orientation data to the encoding of the captured video is done using the same technique as described earlier in the section about adding orientation data to the preview stream.

In the following example, a file is created to which the captured video will be written. An MP4 encoding profile is created using the static method [CreateMp4](#). The proper orientation for the video is obtained from the **CameraRotationHelper** class with a call to **GetCameraCaptureOrientation**. Because the rotation property requires the orientation to be expressed in counterclockwise degrees, the **ConvertSimpleOrientationToClockwiseDegrees** helper method is called to convert the orientation value. Next, create the GUID representing the Media Foundation Transform (MFT) attribute for video stream rotation. In C++ you can use the constant [MF_MT_VIDEO_ROTATION](#), but in C# you must manually specify the GUID value. Add a property value to the stream properties object, specifying the GUID as the key and the rotation as the value. Finally call [StartRecordToStorageFileAsync](#) to begin recording video encoded with orientation data.

C#

```

private async Task StartRecordingWithOrientationAsync()
{
    try
    {
        var videoFile = await
        KnownFolders.VideosLibrary.CreateFileAsync("SimpleVideo.mp4",
        CreationCollisionOption.GenerateUniqueName);

        var encodingProfile =
        MediaEncodingProfile.CreateMp4(VideoEncodingQuality.Auto);

        var rotationAngle =
        CameraRotationHelper.ConvertSimpleOrientationToClockwiseDegrees(
            _rotationHelper.GetCameraCaptureOrientation());
    }
}

```



```

        Guid RotationKey = new Guid("C380465D-2271-428C-9B83-ECEA3B4A85C1");
        encodingProfile.Video.Properties.Add(RotationKey,
        PropertyValue.CreateInt32(rotationAngle));

        await mediaCapture.StartRecordToStorageFileAsync(encodingProfile,
        videoFile);
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine("Exception when starting video
        recording: {0}", ex.ToString());
    }
}

```

CameraRotationHelper full code listing

The following code snippet lists the full code for the **CameraRotationHelper** class that manages the hardware orientation sensors, calculates the proper orientation values for photos and videos, and provides helper methods to convert between the different representations of orientation that are used by different Windows features. If you follow the guidance shown in the article above, you can add this class to your project as-is without having to make any changes. Of course, you can feel free to customize the following code to meet the needs of your particular scenario.

This helper class uses the device's [SimpleOrientationSensor](#) to determine the current orientation of the device chassis and the [DisplayInformation](#) class to determine the current orientation of the display. Each of these classes provide events that are raised when the current orientation changes. The panel on which the capture device is mounted - front-facing, back-facing, or external - is used to determine whether the preview stream should be mirrored. Also, the [EnclosureLocation.RotationAngleInDegreesClockwise](#) property, supported by some devices, is used to determine the orientation in which the camera is mounted on the chassis.

The following methods can be used to get recommended orientation values for the specified camera app tasks:

- **GetUIOrientation** - Returns the suggested orientation for camera UI elements.
- **GetCameraCaptureOrientation** - Returns the suggested orientation for encoding into image metadata.
- **GetCameraPreviewOrientation** - Returns the suggested orientation for the preview stream to provide a natural user experience.

```

class CameraRotationHelper
{
    private EnclosureLocation _cameraEnclosureLocation;
    private DisplayInformation _displayInformation =
DisplayInformation.GetForCurrentView();
    private SimpleOrientationSensor _orientationSensor =
SimpleOrientationSensor.GetDefault();
    public event EventHandler<bool> OrientationChanged;

    public CameraRotationHelper(EnclosureLocation cameraEnclosureLocation)
    {
        _cameraEnclosureLocation = cameraEnclosureLocation;
        if (!IsEnclosureLocationExternal(_cameraEnclosureLocation))
        {
            _orientationSensor.OrientationChanged +=
SimpleOrientationSensor_OrientationChanged;
        }
        _displayInformation.OrientationChanged +=
DisplayInformation_OrientationChanged;
    }

    private void
SimpleOrientationSensor_OrientationChanged(SimpleOrientationSensor sender,
SimpleOrientationSensorOrientationChangedEventArgs args)
    {
        if (args.Orientation != SimpleOrientation.Faceup && args.Orientation
!= SimpleOrientation.Facedown)
        {
            HandleOrientationChanged(false);
        }
    }

    private void DisplayInformation_OrientationChanged(DisplayInformation
sender, object args)
    {
        HandleOrientationChanged(true);
    }

    private void HandleOrientationChanged(bool updatePreviewStreamRequired)
    {
        var handler = OrientationChanged;
        if (handler != null)
        {
            handler(this, updatePreviewStreamRequired);
        }
    }

    public static bool IsEnclosureLocationExternal(EnclosureLocation
enclosureLocation)
    {
        return (enclosureLocation == null || enclosureLocation.Panel ==
Windows.Devices.Enumeration.Panel.Unknown);
    }
}

```

```

private bool IsCameraMirrored()
{
    // Front panel cameras are mirrored by default
    return (_cameraEnclosureLocation.Panel ==
Windows.Devices.Enumeration.Panel.Front);
}

private SimpleOrientation
GetCameraOrientationRelativeToNativeOrientation()
{
    // Get the rotation angle of the camera enclosure
    return
ConvertClockwiseDegreesToSimpleOrientation((int)_cameraEnclosureLocation.Rot
ationAngleInDegreesClockwise);
}

// Gets the rotation to rotate ui elements
public SimpleOrientation GetUIOrientation()
{
    if (IsEnclosureLocationExternal(_cameraEnclosureLocation))
    {
        // Cameras that are not attached to the device do not rotate
along with it, so apply no rotation
        return SimpleOrientation.NotRotated;
    }

    // Return the difference between the orientation of the device and
the orientation of the app display
    var deviceOrientation = _orientationSensor.GetCurrentOrientation();
    var displayOrientation =
ConvertDisplayOrientationToSimpleOrientation(_displayInformation.CurrentOrie
ntation);
    return SubOrientations(displayOrientation, deviceOrientation);
}

// Gets the rotation of the camera to rotate pictures/videos when saving
to file
public SimpleOrientation GetCameraCaptureOrientation()
{
    if (IsEnclosureLocationExternal(_cameraEnclosureLocation))
    {
        // Cameras that are not attached to the device do not rotate
along with it, so apply no rotation
        return SimpleOrientation.NotRotated;
    }

    // Get the device orientation offset by the camera hardware offset
    var deviceOrientation = _orientationSensor.GetCurrentOrientation();
    var result = SubOrientations(deviceOrientation,
GetCameraOrientationRelativeToNativeOrientation());

    // If the preview is being mirrored for a front-facing camera, then
the rotation should be inverted
    if (IsCameraMirrored())
    {

```

```

        result = MirrorOrientation(result);
    }
    return result;
}

// Gets the rotation of the camera to display the camera preview
public SimpleOrientation GetCameraPreviewOrientation()
{
    if (IsEnclosureLocationExternal(_cameraEnclosureLocation))
    {
        // Cameras that are not attached to the device do not rotate
        // along with it, so apply no rotation
        return SimpleOrientation.NotRotated;
    }

    // Get the app display rotation offset by the camera hardware offset
    var result =
        ConvertDisplayOrientationToSimpleOrientation(_displayInformation.CurrentOrientation);
    result = SubOrientations(result,
        GetCameraOrientationRelativeToNativeOrientation());

    // If the preview is being mirrored for a front-facing camera, then
    // the rotation should be inverted
    if (IsCameraMirrored())
    {
        result = MirrorOrientation(result);
    }
    return result;
}

public static PhotoOrientation
ConvertSimpleOrientationToPhotoOrientation(SimpleOrientation orientation)
{
    switch (orientation)
    {
        case SimpleOrientation.Rotated90DegreesCounterclockwise:
            return PhotoOrientation.Rotate90;
        case SimpleOrientation.Rotated180DegreesCounterclockwise:
            return PhotoOrientation.Rotate180;
        case SimpleOrientation.Rotated270DegreesCounterclockwise:
            return PhotoOrientation.Rotate270;
        case SimpleOrientation.NotRotated:
        default:
            return PhotoOrientation.Normal;
    }
}

public static int
ConvertSimpleOrientationToClockwiseDegrees(SimpleOrientation orientation)
{
    switch (orientation)
    {
        case SimpleOrientation.Rotated90DegreesCounterclockwise:
            return 270;
    }
}

```

```

        case SimpleOrientation.Rotated180DegreesCounterclockwise:
            return 180;
        case SimpleOrientation.Rotated270DegreesCounterclockwise:
            return 90;
        case SimpleOrientation.NotRotated:
        default:
            return 0;
    }
}

private SimpleOrientation
ConvertDisplayOrientationToSimpleOrientation(DisplayOrientations
orientation)
{
    SimpleOrientation result;
    switch (orientation)
    {
        case DisplayOrientations.Landscape:
            result = SimpleOrientation.NotRotated;
            break;
        case DisplayOrientations.PortraitFlipped:
            result = SimpleOrientation.Rotated90DegreesCounterclockwise;
            break;
        case DisplayOrientations.LandscapeFlipped:
            result =
SimpleOrientation.Rotated180DegreesCounterclockwise;
            break;
        case DisplayOrientations.Portrait:
        default:
            result =
SimpleOrientation.Rotated270DegreesCounterclockwise;
            break;
    }

    // Above assumes landscape; offset is needed if native orientation
    is portrait
    if (_displayInformation.NativeOrientation ==
DisplayOrientations.Portrait)
    {
        result = AddOrientations(result,
SimpleOrientation.Rotated90DegreesCounterclockwise);
    }

    return result;
}

private static SimpleOrientation MirrorOrientation(SimpleOrientation
orientation)
{
    // This only affects the 90 and 270 degree cases, because rotating 0
    and 180 degrees is the same clockwise and counter-clockwise
    switch (orientation)
    {
        case SimpleOrientation.Rotated90DegreesCounterclockwise:
            return SimpleOrientation.Rotated270DegreesCounterclockwise;

```

```

        case SimpleOrientation.Rotated270DegreesCounterclockwise:
            return SimpleOrientation.Rotated90DegreesCounterclockwise;
    }
    return orientation;
}

private static SimpleOrientation AddOrientations(SimpleOrientation a,
SimpleOrientation b)
{
    var aRot = ConvertSimpleOrientationToClockwiseDegrees(a);
    var bRot = ConvertSimpleOrientationToClockwiseDegrees(b);
    var result = (aRot + bRot) % 360;
    return ConvertClockwiseDegreesToSimpleOrientation(result);
}

private static SimpleOrientation SubOrientations(SimpleOrientation a,
SimpleOrientation b)
{
    var aRot = ConvertSimpleOrientationToClockwiseDegrees(a);
    var bRot = ConvertSimpleOrientationToClockwiseDegrees(b);
    //add 360 to ensure the modulus operator does not operate on a
negative
    var result = (360 + (aRot - bRot)) % 360;
    return ConvertClockwiseDegreesToSimpleOrientation(result);
}

private static VideoRotation
ConvertSimpleOrientationToVideoRotation(SimpleOrientation orientation)
{
    switch (orientation)
    {
        case SimpleOrientation.Rotated90DegreesCounterclockwise:
            return VideoRotation.Clockwise270Degrees;
        case SimpleOrientation.Rotated180DegreesCounterclockwise:
            return VideoRotation.Clockwise180Degrees;
        case SimpleOrientation.Rotated270DegreesCounterclockwise:
            return VideoRotation.Clockwise90Degrees;
        case SimpleOrientation.NotRotated:
        default:
            return VideoRotation.None;
    }
}

private static SimpleOrientation
ConvertClockwiseDegreesToSimpleOrientation(int orientation)
{
    switch (orientation)
    {
        case 270:
            return SimpleOrientation.Rotated90DegreesCounterclockwise;
        case 180:
            return SimpleOrientation.Rotated180DegreesCounterclockwise;
        case 90:
            return SimpleOrientation.Rotated270DegreesCounterclockwise;
        case 0:

```

```
        default:
            return SimpleOrientation.NotRotated;
    }
}
```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Discover and select camera capabilities with camera profiles

Article • 10/20/2022

This article discusses how to use camera profiles to discover and manage the capabilities of different video capture devices. This includes tasks such as selecting profiles that support specific resolutions or frame rates, profiles that support simultaneous access to multiple cameras, and profiles that support HDR.

ⓘ Note

This article builds on concepts and code discussed in [Basic photo, video, and audio capture with MediaCapture](#), which describes the steps for implementing basic photo and video capture. It is recommended that you familiarize yourself with the basic media capture pattern in that article before moving on to more advanced capture scenarios. The code in this article assumes that your app already has an instance of MediaCapture that has been properly initialized.

About camera profiles

Cameras on different devices support different capabilities including the set of supported capture resolutions, frame rate for video captures, and whether HDR or variable frame rate captures are supported. The Universal Windows Platform (UWP) media capture framework stores this set of capabilities in a [MediaCaptureVideoProfileMediaDescription](#). A camera profile, represented by a [MediaCaptureVideoProfile](#) object, has three collections of media descriptions; one for photo capture, one for video capture, and another for video preview.

Before initializing your [MediaCapture](#) object, you can query the capture devices on the current device to see what profiles are supported. When you select a supported profile, you know that the capture device supports all of the capabilities in the profile's media descriptions. This eliminates the need for a trial and error approach to determining which combinations of capabilities are supported on a particular device.

C#

```
var mediaInitSettings = new MediaCaptureInitializationSettings {
```



```
VideoDeviceId = cameraDevice.Id };
```

The code examples in this article replace this minimal initialization with the discovery of camera profiles supporting various capabilities, which are then used to initialize the media capture device.

Find a video device that supports camera profiles

Before searching for supported camera profiles, you should find a capture device that supports the use of camera profiles. The **GetVideoProfileSupportedDeviceIdAsync** helper method defined in the example below uses the [DeviceInformation.FindAllAsync](#) method to retrieve a list of all available video capture devices. It loops through all of the devices in the list, calling the static method, [IsVideoProfileSupported](#), for each device to see if it supports video profiles. Also, the [EnclosureLocation.Panel](#) property for each device, allowing you to specify whether you want a camera on the front or back of the device.

If a device that supports camera profiles is found on the specified panel, the **Id** value, containing the device's ID string, is returned.

C#

```
public async Task<string>
GetVideoProfileSupportedDeviceIdAsync(Windows.Devices.Enumeration.Panel
panel)
{
    string deviceId = string.Empty;

    // Finds all video capture devices
    DeviceInformationCollection devices = await
DeviceInformation.FindAllAsync(DeviceClass.VideoCapture);

    foreach (var device in devices)
    {
        // Check if the device on the requested panel supports Video Profile
        if (MediaCapture.IsVideoProfileSupported(device.Id) &&
device.EnclosureLocation.Panel == panel)
        {
            // We've located a device that supports Video Profiles on
expected panel
            deviceId = device.Id;
            break;
        }
    }
}
```

```
        return deviceId;  
    }
```

If the device ID returned from the **GetVideoProfileSupportedDeviceIdAsync** helper method is null or an empty string, there is no device on the specified panel that supports camera profiles. In this case, you should initialize your media capture device without using profiles.

C#

```
string videoDeviceId = await  
GetVideoProfileSupportedDeviceIdAsync(Windows.Devices.Enumeration.Panel.Back  
);  
  
if (string.IsNullOrEmpty(videoDeviceId))  
{  
    // No devices on the specified panel support video profiles. .  
    return;  
}
```

Select a profile based on supported resolution and frame rate

To select a profile with particular capabilities, such as with the ability to achieve a particular resolution and frame rate, you should first call the helper method defined above to get the ID of a capture device that supports using camera profiles.

Create a new **MediaCaptureInitializationSettings** object, passing in the selected device ID. Next, call the static method **MediaCapture.FindAllVideoProfiles** to get a list of all camera profiles supported by the device.

This example uses a Linq query method, included in the using **System.Linq** namespace, to select a profile that contains a **SupportedRecordMediaDescription** object where the **Width**, **Height**, and **FrameRate** properties match the requested values. If a match is found, **VideoProfile** and **RecordMediaDescription** of the **MediaCaptureInitializationSettings** are set to the values from the anonymous type returned from the Linq query. If no match is found, the default profile is used.

C#

```
var mediaInitSettings = new MediaCaptureInitializationSettings {  
    VideoDeviceId = videoDeviceId };
```

```

IEnumerable<MediaCaptureVideoProfile> profiles =
MediaCapture.FindAllVideoProfiles(videoDeviceId);

var match = (from profile in profiles
             from desc in profile.SupportedRecordMediaDescription
             where desc.Width == 640 && desc.Height == 480 &&
Math.Round(desc.FrameRate) == 30
             select new { profile, desc }).FirstOrDefault();

if (match != null)
{
    mediaInitSettings.VideoProfile = match.profile;
    mediaInitSettings.RecordMediaDescription = match.desc;
}
else
{
    // Could not locate a WVGA 30FPS profile, use default video recording
    profile
    mediaInitSettings.VideoProfile = profiles[0];
}

```

After you populate the **MediaCaptureInitializationSettings** with your desired camera profile, you simply call **InitializeAsync** on your media capture object to configure it to the desired profile.

C#

```
await _mediaCapture.InitializeAsync(mediaInitSettings);
```

Use media frame source groups to get profiles

Starting with Windows 10, version 1803, you can use the **MediaFrameSourceGroup** class to get camera profiles with specific capabilities before initializing the **MediaCapture** object. Frame source groups allow device manufacturers to represent groups of sensors or capture capabilities as a single virtual device. This enables computational photography scenarios such as using depth and color cameras together, but can also be used to select camera profiles for simple capture scenarios. For more information on using **MediaFrameSourceGroup**, see [Process media frames with MediaFrameReader](#).

The example method below shows how to use **MediaFrameSourceGroup** objects to find a camera profile that supports a known video profile, such as one that supports HDR or variable photo sequence. First, call **MediaFrameSourceGroup.FindAllAsync** to get a list of all media frame source groups available on the current device. Loop through each source group and call **MediaCapture.FindKnownVideoProfiles** to get a list of all of the video profiles for the current source group that support the specified profile, in this case

HDR with WCG photo. If a profile that meets the criteria is found, create a new **MediaCaptureInitializationSettings** object and set the **VideoProfile** to the select profile and the **VideoDeviceId** to the **Id** property of the current media frame source group. So, for example, you could pass the value **KnownVideoProfile.HdrWithWcgVideo** into this method to get media capture settings that support HDR video. Pass **KnownVideoProfile.VariablePhotoSequence** to get settings that support variable photo sequence.

C#

```
private async Task<MediaCaptureInitializationSettings>
GetKnownVideoProfile(KnownVideoProfile knownVideoProfile)
{
    IReadOnlyList<MediaFrameSourceGroup> sourceGroups = await
MediaFrameSourceGroup.FindAllAsync();
    MediaCaptureInitializationSettings settings = null;

    foreach (MediaFrameSourceGroup sg in sourceGroups)
    {
        // Find a device that support VariablePhotoSequence
        IReadOnlyList<MediaCaptureVideoProfile> profileList =
MediaCapture.FindKnownVideoProfiles(
                                sg.Id,
                                knownVideoProfile); // e.g.
KnownVideoProfile.HdrWithWcgVideo

        if (profileList.Count > 0)
        {
            settings = new MediaCaptureInitializationSettings();
            settings.VideoProfile = profileList[0];
            settings.VideoDeviceId = sg.Id;
            break;
        }
    }
    return settings;
}
```

Use known profiles to find a profile that supports HDR video (legacy technique)

ⓘ Note

The APIs described in this section are deprecated starting with Windows 10, version 1803. See the previous section, **Use media frame source groups to get profiles**.

Selecting a profile that supports HDR begins like the other scenarios. Create a **MediaCaptureInitializationSettings** and a string to hold the capture device ID. Add a boolean variable that will track whether HDR video is supported.

C#

```
MediaCaptureInitializationSettings mediaInitSettings = new
MediaCaptureInitializationSettings();
string videoDeviceId = string.Empty;
bool HdrVideoSupported = false;
```

Use the **GetVideoProfileSupportedDeviceIdAsync** helper method defined above to get the device ID for a capture device that supports camera profiles.

C#

```
// Select the first video capture device found on the back of the device
videoDeviceId = await
GetVideoProfileSupportedDeviceIdAsync(Windows.Devices.Enumeration.Panel.Back
);

if (string.IsNullOrEmpty(videoDeviceId))
{
    // No devices on the specified panel support video profiles. .
    return;
}
```

The static method **MediaCapture.FindKnownVideoProfiles** returns the camera profiles supported by the specified device that is categorized by the specified **KnownVideoProfile** value. For this scenario, the **VideoRecording** value is specified to limit the returned camera profiles to ones that support video recording.

Loop through the returned list of camera profiles. For each camera profile, loop through each **VideoProfileMediaDescription** in the profile checking to see if the **IsHdrVideoSupported** property is true. After a suitable media description is found, break out of the loop and assign the profile and description objects to the **MediaCaptureInitializationSettings** object.

C#

```
IReadOnlyList<MediaCaptureVideoProfile> profiles =
    MediaCapture.FindKnownVideoProfiles(videoDeviceId,
KnownVideoProfile.VideoRecording);

// Walk through available profiles, look for first profile with HDR
supported Video Profile
foreach (MediaCaptureVideoProfile profile in profiles)
```

```

{
    IReadOnlyList<MediaCaptureVideoProfileMediaDescription>
recordMediaDescription =
    profile.SupportedRecordMediaDescription;
    foreach (MediaCaptureVideoProfileMediaDescription
videoProfileMediaDescription in recordMediaDescription)
    {
        if (videoProfileMediaDescription.IsHdrVideoSupported)
        {
            // We've located the profile and description for HDR Video, set
profile and flag
            mediaInitSettings.VideoProfile = profile;
            mediaInitSettings.RecordMediaDescription =
videoProfileMediaDescription;
            HdrVideoSupported = true;
            break;
        }
    }

    if (HdrVideoSupported)
    {
        // Profile with HDR support found. Stop looking.
        break;
    }
}

```

Determine if a device supports simultaneous photo and video capture

Many devices support capturing photos and video simultaneously. To determine if a capture device supports this, call [MediaCapture.FindAllVideoProfiles](#) to get all of the camera profiles supported by the device. Use a link query to find a profile that has at least one entry for both [SupportedPhotoMediaDescription](#) and [SupportedRecordMediaDescription](#) which means that the profile supports simultaneous capture.

C#

```

var simultaneousPhotoAndVideoSupported = false;

IReadOnlyList<MediaCaptureVideoProfile> profiles =
MediaCapture.FindAllVideoProfiles(videoDeviceId);

var match = (from profile in profiles
              where profile.SupportedPhotoMediaDescription.Any() &&
                    profile.SupportedRecordMediaDescription.Any()
              select profile).FirstOrDefault();

if (match != null)

```

```
{  
    // Simultaneous photo and video supported  
    simultaneousPhotoAndVideoSupported = true;  
}  
else  
{  
    // Simultaneous photo and video not supported  
    simultaneousPhotoAndVideoSupported = false;  
}
```

You can refine this query to look for profiles that support specific resolutions or other capabilities in addition to simultaneous video record. You can also use the [MediaCapture.FindKnownVideoProfiles](#) and specify the [BalancedVideoAndPhoto](#) value to retrieve profiles that support simultaneous capture, but querying all profiles will provide more complete results.

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Set format, resolution, and frame rate for MediaCapture

Article • 10/20/2022

This article shows you how to use the [IMediaEncodingProperties](#) interface to set the resolution and frame rate of the camera preview stream and captured photos and video. It also shows how to ensure that the aspect ratio of the preview stream matches that of the captured media.

Camera profiles offer a more advanced way of discovering and setting the stream properties of the camera, but they are not supported for all devices. For more information, see [Camera profiles](#).

The code in this article was adapted from the [CameraResolution sample](#). You can download the sample to see the code used in context or to use the sample as a starting point for your own app.

ⓘ Note

This article builds on concepts and code discussed in [Basic photo, video, and audio capture with MediaCapture](#), which describes the steps for implementing basic photo and video capture. It is recommended that you familiarize yourself with the basic media capture pattern in that article before moving on to more advanced capture scenarios. The code in this article assumes that your app already has an instance of MediaCapture that has been properly initialized.

A media encoding properties helper class

Creating a simple helper class to wrap the functionality of the [IMediaEncodingProperties](#) interface makes it easier to select a set of encoding properties that meet particular criteria. This helper class is particularly useful due to the following behavior of the encoding properties feature:

Warning The [VideoDeviceController.GetAvailableMediaStreamProperties](#) method takes a member of the [MediaStreamType](#) enumeration, such as [VideoRecord](#) or [Photo](#), and returns a list of either [ImageEncodingProperties](#) or [VideoEncodingProperties](#) objects that convey the stream encoding settings, such as the resolution of the captured photo or video. The results of calling [GetAvailableMediaStreamProperties](#) may include [ImageEncodingProperties](#) or [VideoEncodingProperties](#) regardless of what

MediaStreamType value is specified. For this reason, you should always check the type of each returned value and cast it to the appropriate type before attempting to access any of the property values.

The helper class defined below handles the type checking and casting for [ImageEncodingProperties](#) or [VideoEncodingProperties](#) so that your app code doesn't need to distinguish between the two types. In addition to this, the helper class exposes properties for the aspect ratio of the properties, the frame rate (for video encoding properties only), and a friendly name that makes it easier to display the encoding properties in the app's UI.

You must include the [Windows.Media.MediaProperties](#) namespace in the source file for the helper class.

C#

```
using Windows.Media.MediaProperties;
using Windows.Media.Capture.Frames;
```

C#

```
class StreamPropertiesHelper
{
    private IMediaEncodingProperties _properties;

    public StreamPropertiesHelper(IMediaEncodingProperties properties)
    {
        if (properties == null)
        {
            throw new ArgumentNullException(nameof(properties));
        }

        // This helper class only uses VideoEncodingProperties or
        VideoEncodingProperties
        if (!(properties is ImageEncodingProperties) && !(properties is
VideoEncodingProperties))
        {
            throw new ArgumentException("Argument is of the wrong type.
Required: " + typeof(ImageEncodingProperties).Name
            + " or " + typeof(VideoEncodingProperties).Name + ".",
nameof(properties));
        }

        // Store the actual instance of the IMediaEncodingProperties for
setting them later
        _properties = properties;
    }

    public uint Width
```

```

{
    get
    {
        if (_properties is ImageEncodingProperties)
        {
            return (_properties as ImageEncodingProperties).Width;
        }
        else if (_properties is VideoEncodingProperties)
        {
            return (_properties as VideoEncodingProperties).Width;
        }

        return 0;
    }
}

public uint Height
{
    get
    {
        if (_properties is ImageEncodingProperties)
        {
            return (_properties as ImageEncodingProperties).Height;
        }
        else if (_properties is VideoEncodingProperties)
        {
            return (_properties as VideoEncodingProperties).Height;
        }

        return 0;
    }
}

public uint FrameRate
{
    get
    {
        if (_properties is VideoEncodingProperties)
        {
            if ((_properties as
VideoEncodingProperties).FrameRate.Denominator != 0)
            {
                return (_properties as
VideoEncodingProperties).FrameRate.Numerator /
                    (_properties as
VideoEncodingProperties).FrameRate.Denominator;
            }

            return 0;
        }
    }
}

public double AspectRatio
{

```

```

        get { return Math.Round((Height != 0) ? (Width / (double)Height) :
double.NaN, 2); }
    }

    public IMediaEncodingProperties EncodingProperties
    {
        get { return _properties; }
    }

    public string GetFriendlyName(bool showFrameRate = true)
    {
        if (_properties is ImageEncodingProperties ||
            !showFrameRate)
        {
            return Width + "x" + Height + " [" + AspectRatio + "]" +
                _properties.Subtype;
        }
        else if (_properties is VideoEncodingProperties)
        {
            return Width + "x" + Height + " [" + AspectRatio + "]" +
                FrameRate + "FPS " + _properties.Subtype;
        }

        return String.Empty;
    }
}

```

Determine if the preview and capture streams are independent

On some devices, the same hardware pin is used for both preview and capture streams. On these devices, setting the encoding properties of one will also set the other. On devices that use different hardware pins for capture and preview, the properties can be set for each stream independently. Use the following code to determine if the preview and capture streams are independent. You should adjust your UI to enable or disable the setting of the streams independently based on the result of this test.

C#

```

private void CheckIfStreamsAreIdentical()
{
    if (_mediaCapture.MediaCaptureSettings.VideoDeviceCharacteristic ==
        VideoDeviceCharacteristic.AllStreamsIdentical ||
        _mediaCapture.MediaCaptureSettings.VideoDeviceCharacteristic ==
        VideoDeviceCharacteristic.PreviewRecordStreamsIdentical)
    {
        ShowMessageToUser("Preview and video streams for this device are
            identical. Changing one will affect the other");
    }
}

```

```
}  
}
```

Get a list of available stream properties

Get a list of the available stream properties for a capture device by getting the [VideoDeviceController](#) for your app's [MediaCapture](#) object and then calling [GetAvailableMediaStreamProperties](#) and passing in one of the [MediaStreamType](#) values, **VideoPreview**, **VideoRecord**, or **Photo**. In this example, Linq syntax is used to create a list of **StreamPropertiesHelper** objects, defined previously in this article, for each of the [IMediaEncodingProperties](#) values returned from [GetAvailableMediaStreamProperties](#). This example first uses Linq extension methods to order the returned properties based first on resolution and then on frame rate.

If your app has specific resolution or frame rate requirements, you can select a set of media encoding properties programmatically. A typical camera app will instead expose the list of available properties in the UI and allow the user to select their desired settings. A **ComboBoxItem** is created for each item in the list of **StreamPropertiesHelper** objects in the list. The content is set to the friendly name returned by the helper class and the tag is set to the helper class itself so it can be used later to retrieve the associated encoding properties. Each **ComboBoxItem** is then added to the **ComboBox** passed into the method.

C#

```
private void PopulateStreamPropertiesUI(MediaStreamType streamType, ComboBox  
comboBox, bool showFrameRate = true)  
{  
    // Query all properties of the specified stream type  
    IEnumerable<StreamPropertiesHelper> allStreamProperties =  
  
    _mediaCapture.VideoDeviceController.GetAvailableMediaStreamProperties(stream  
Type).Select(x => new StreamPropertiesHelper(x));  
  
    // Order them by resolution then frame rate  
    allStreamProperties = allStreamProperties.OrderByDescending(x =>  
x.Height * x.Width).ThenByDescending(x => x.FrameRate);  
  
    // Populate the combo box with the entries  
    foreach (var property in allStreamProperties)  
    {  
        ComboBoxItem comboBoxItem = new ComboBoxItem();  
        comboBoxItem.Content = property.GetFriendlyName(showFrameRate);  
        comboBoxItem.Tag = property;  
        comboBox.Items.Add(comboBoxItem);  
    }  
}
```

```
}  
}
```

Set the desired stream properties

Tell the video device controller to use your desired encoding properties by calling [SetMediaStreamPropertiesAsync](#), passing in the **MediaStreamType** value indicating whether the photo, video, or preview properties should be set. This example sets the requested encoding properties when the user selects an item in one of the **ComboBox** objects populated with the **PopulateStreamPropertiesUI** helper method.

C#

```
private async void PreviewSettings_Changed(object sender, RoutedEventArgs e)  
{  
    if (_isPreviewing)  
    {  
        var selectedItem = (sender as ComboBox).SelectedItem as  
ComboBoxItem;  
        var encodingProperties = (selectedItem.Tag as  
StreamPropertiesHelper).EncodingProperties;  
        await  
_mediaCapture.VideoDeviceController.SetMediaStreamPropertiesAsync(MediaStrea  
mType.VideoPreview, encodingProperties);  
    }  
}
```

C#

```
private async void PhotoSettings_Changed(object sender, RoutedEventArgs e)  
{  
    if (_isPreviewing)  
    {  
        var selectedItem = (sender as ComboBox).SelectedItem as  
ComboBoxItem;  
        var encodingProperties = (selectedItem.Tag as  
StreamPropertiesHelper).EncodingProperties;  
        await  
_mediaCapture.VideoDeviceController.SetMediaStreamPropertiesAsync(MediaStrea  
mType.Photo, encodingProperties);  
    }  
}
```

C#

```
private async void VideoSettings_Changed(object sender, RoutedEventArgs e)  
{
```

```

        if (!_isPreviewing)
        {
            var selectedItem = (sender as ComboBox).SelectedItem as
ComboBoxItem;
            var encodingProperties = (selectedItem.Tag as
StreamPropertiesHelper).EncodingProperties;
            await
_mediaCapture.VideoDeviceController.SetMediaStreamPropertiesAsync(MediaStrea
mType.VideoRecord, encodingProperties);
        }
    }
}

```

Match the aspect ratio of the preview and capture streams

A typical camera app will provide UI for the user to select the video or photo capture resolution but will programmatically set the preview resolution. There are a few different strategies for selecting the best preview stream resolution for your app:

- Select the highest available preview resolution, letting the UI framework perform any necessary scaling of the preview.
- Select the preview resolution closest to the capture resolution so that the preview displays the closest representation to the final captured media.
- Select the preview resolution closest to the size of the [CaptureElement](#) so that no more pixels than necessary are going through the preview stream pipeline.

Important It is possible, on some devices, to set a different aspect ratio for the camera's preview stream and capture stream. Frame cropping caused by this mismatch can result in content being present in the captured media that was not visible in the preview which can result in a negative user experience. It is strongly recommended that you use the same aspect ratio, within a small tolerance window, for the preview and capture streams. It is fine to have entirely different resolutions enabled for capture and preview as long as the aspect ratio match closely.

To ensure that the photo or video capture streams match the aspect ratio of the preview stream, this example calls [VideoDeviceController.GetMediaStreamProperties](#) and passes in the **VideoPreview** enum value to request the current stream properties for the preview stream. Next a small aspect ratio tolerance window is defined so that we can include aspect ratios that are not exactly the same as the preview stream, as long as they are close. Next, a Linq extension method is used to select just the **StreamPropertiesHelper** objects where the aspect ratio is within the defined tolerance range of the preview stream.

C#

```
private void MatchPreviewAspectRatio(MediaStreamType streamType, ComboBox
comboBox)
{
    // Query all properties of the specified stream type
    IEnumerable<StreamPropertiesHelper> allVideoProperties =

_mediaCapture.VideoDeviceController.GetAvailableMediaStreamProperties(stream
Type).Select(x => new StreamPropertiesHelper(x));

    // Query the current preview settings
    StreamPropertiesHelper previewProperties = new
StreamPropertiesHelper(_mediaCapture.VideoDeviceController.GetMediaStreamPro
perties(MediaStreamType.VideoPreview));

    // Get all formats that have the same-ish aspect ratio as the preview
    // Allow for some tolerance in the aspect ratio comparison
    const double ASPECT_RATIO_TOLERANCE = 0.015;
    var matchingFormats = allVideoProperties.Where(x =>
Math.Abs(x.AspectRatio - previewProperties.AspectRatio) <
ASPECT_RATIO_TOLERANCE);

    // Order them by resolution then frame rate
    allVideoProperties = matchingFormats.OrderByDescending(x => x.Height *
x.Width).ThenByDescending(x => x.FrameRate);

    // Clear out old entries and populate the video combo box with new
matching entries
    comboBox.Items.Clear();
    foreach (var property in allVideoProperties)
    {
        ComboBoxItem comboBoxItem = new ComboBoxItem();
        comboBoxItem.Content = property.GetFriendlyName();
        comboBoxItem.Tag = property;
        comboBox.Items.Add(comboBoxItem);
    }
    comboBox.SelectedIndex = -1;
}
```

High dynamic range (HDR) and low-light photo capture

Article • 10/20/2022

This article shows you how to use the [AdvancedPhotoCapture](#) class to capture high dynamic range (HDR) photos. This API also allows you to obtain a reference frame from the HDR capture before the processing of the final image is complete.

Other articles related to HDR capture include:

- You can use the [SceneAnalysisEffect](#) to allow the system to evaluate the content of the media capture preview stream to determine if HDR processing would improve the capture result. For more information, see [Scene analysis for MediaCapture](#).
- Use the [HdrVideoControl](#) to capture video using the Windows built-in HDR processing algorithm. For more information, see [Capture device controls for video capture](#).
- You can use the [VariablePhotoSequenceCapture](#) to capture a sequence of photos, each with different capture settings, and implement your own HDR or other processing algorithm. For more information, see [Variable photo sequence](#).

ⓘ Note

Starting with Windows 10, version 1709, recording video and using **AdvancedPhotoCapture** concurrently is supported. This is not supported in previous versions. This change means that you can have a prepared **LowLagMediaRecording** and **AdvancedPhotoCapture** at the same time. You can start or stop video recording between calls to **MediaCapture.PrepareAdvancedPhotoCaptureAsync** and **AdvancedPhotoCapture.FinishAsync**. You can also call **AdvancedPhotoCapture.CaptureAsync** while video is recording. However, some **AdvancedPhotoCapture** scenarios, like capturing an HDR photo while recording video would cause some video frames to be altered by the HDR capture, resulting in a negative user experience. For this reason, the list of modes returned by the **AdvancedPhotoControl.SupportedModes** will be different while video is recording. You should check this value immediately after starting or stopping video recording to ensure that the desired mode is supported in the current video recording state.

ⓘ Note

Starting with Windows 10, version 1709, when the **AdvancedPhotoCapture** is set to HDR mode, the setting of the **FlashControl.Enabled** property is ignored and the flash is never fired. For other capture modes, if the **FlashControl.Enabled**, it will override the **AdvancedPhotoCapture** settings and cause a normal photo to be captured with flash. If **Auto** is set to true, the **AdvancedPhotoCapture** may or may not use flash, depending on the camera driver's default behavior for the conditions in the current scene. On previous releases, the **AdvancedPhotoCapture** flash setting always overrides the **FlashControl.Enabled** setting.

ⓘ Note

This article builds on concepts and code discussed in **Basic photo, video, and audio capture with MediaCapture**, which describes the steps for implementing basic photo and video capture. We recommend that you familiarize yourself with the basic media capture pattern in that article before moving on to more advanced capture scenarios. The code in this article assumes that your app already has an instance of **MediaCapture** that has been properly initialized.

There is a Universal Windows Sample demonstrating the use of the **AdvancedPhotoCapture** class that you can use to see the API used in context or as a starting point for your own app. For more information see, [Camera Advanced Capture sample](#) [↗].

Advanced photo capture namespaces

The code examples in this article use APIs in the following namespaces in addition to the namespaces required for basic media capture.

C#

```
using Windows.Media.Core;  
using Windows.Media.Devices;
```

HDR photo capture

Determine if HDR photo capture is supported on the current device

The HDR capture technique described in this article is performed using the [AdvancedPhotoCapture](#) object. Not all devices support HDR capture with [AdvancedPhotoCapture](#). Determine if the device on which your app is currently running supports the technique by getting the [MediaCapture](#) object's [VideoDeviceController](#) and then getting the [AdvancedPhotoControl](#) property. Check the video device controller's [SupportedModes](#) collection to see if it includes [AdvancedPhotoMode.Hdr](#). If it does, HDR capture using [AdvancedPhotoCapture](#) is supported.

C#

```
bool _hdrSupported;
private void IsHdrPhotoSupported()
{
    _hdrSupported =
        _mediaCapture.VideoDeviceController.AdvancedPhotoControl.SupportedModes.Contains(Windows.Media.Devices.AdvancedPhotoMode.Hdr);
}
```

Configure and prepare the AdvancedPhotoCapture object

Because you will need to access the [AdvancedPhotoCapture](#) instance from multiple places within your code, you should declare a member variable to hold the object.

C#

```
private AdvancedPhotoCapture _advancedCapture;
```

In your app, after you have initialized the [MediaCapture](#) object, create an [AdvancedPhotoCaptureSettings](#) object and set the mode to [AdvancedPhotoMode.Hdr](#). Call the [AdvancedPhotoControl](#) object's [Configure](#) method, passing in the [AdvancedPhotoCaptureSettings](#) object you created.

Call the [MediaCapture](#) object's [PrepareAdvancedPhotoCaptureAsync](#), passing in an [ImageEncodingProperties](#) object specifying the type of encoding the capture should use. The [ImageEncodingProperties](#) class provides static methods for creating the image encodings that are supported by [MediaCapture](#).

[PrepareAdvancedPhotoCaptureAsync](#) returns the [AdvancedPhotoCapture](#) object you will use to initiate photo capture. You can use this object to register handlers for the

[OptionalReferencePhotoCaptured](#) and [AllPhotosCaptured](#) which are discussed later in this article.

C#

```
if (_hdrSupported == false) return;

// Choose HDR mode
var settings = new AdvancedPhotoCaptureSettings { Mode =
AdvancedPhotoMode.Hdr };

// Configure the mode
_mediaCapture.VideoDeviceController.AdvancedPhotoControl.Configure(settings)
;

// Prepare for an advanced capture
_advancedCapture =
    await
_mediaCapture.PrepareAdvancedPhotoCaptureAsync(ImageEncodingProperties.Create
eUncompressed(MediaPixelFormat.Nv12));

// Register for events published by the AdvancedCapture
_advancedCapture.AllPhotosCaptured += AdvancedCapture_AllPhotosCaptured;
_advancedCapture.OptionalReferencePhotoCaptured +=
AdvancedCapture_OptionalReferencePhotoCaptured;
```

Capture an HDR photo

Capture an HDR photo by calling the [AdvancedPhotoCapture](#) object's [CaptureAsync](#) method. This method returns an [AdvancedCapturedPhoto](#) object that provides the captured photo in its [Frame](#) property.

C#

```
try
{
    // Start capture, and pass the context object
    AdvancedCapturedPhoto advancedCapturedPhoto = await
_advancedCapture.CaptureAsync();

    using (var frame = advancedCapturedPhoto.Frame)
    {
        // Read the current orientation of the camera and the capture time
        var photoOrientation =
CameraRotationHelper.ConvertSimpleOrientationToPhotoOrientation(
    _rotationHelper.GetCameraCaptureOrientation());
        var fileName = String.Format("SimplePhoto_{0}_HDR.jpg",
DateTime.Now.ToString("HHmmss"));
        await SaveCapturedFrameAsync(frame, fileName, photoOrientation);
    }
}
```

```

    }
}
catch (Exception ex)
{
    Debug.WriteLine("Exception when taking an HDR photo: {0}",
ex.ToString());
}

```

Most photography apps will want to encode a captured photo's rotation into the image file so that it can be displayed correctly by other apps and devices. This example shows the use of the helper class **CameraRotationHelper** to calculate the proper orientation for the file. This class is described and listed in full in the article [Handle device orientation with MediaCapture](#).

The **SaveCapturedFrameAsync** helper method, which saves the image to disk, is discussed later in this article.

Get optional reference frame

The HDR process captures multiple frames and then composites them into a single image after all of the frames have been captured. You can get access to a frame after it is captured but before the entire HDR process is complete by handling the **OptionalReferencePhotoCaptured** event. You don't need to do this if you are only interested in the final HDR photo result.

Important

OptionalReferencePhotoCaptured is not raised on devices that support hardware HDR and therefore do not generate reference frames. Your app should handle the case where this event is not raised.

Because the reference frame arrives out of context of the call to **CaptureAsync**, a mechanism is provided to pass context information to the **OptionalReferencePhotoCaptured** handler. First you should call an object that will contain your context information. The name and contents of this object is up to you. This example defines an object that has members to track the file name and camera orientation of the capture.

```

C#

public class MyAdvancedCaptureContextObject
{
    public string CaptureFileName;

```

```
public PhotoOrientation CaptureOrientation;
}
```

Create a new instance of your context object, populate its members, and then pass it into the overload of **CaptureAsync** that accepts an object as a parameter.

C#

```
// Read the current orientation of the camera and the capture time
var photoOrientation =
    CameraRotationHelper.ConvertSimpleOrientationToPhotoOrientation(
        _rotationHelper.GetCameraCaptureOrientation());
var fileName = String.Format("SimplePhoto_{0}_HDR.jpg",
    DateTime.Now.ToString("HHmmss"));

// Create a context object, to identify the capture in the
OptionalReferencePhotoCaptured event
var context = new MyAdvancedCaptureContextObject()
{
    CaptureFileName = fileName,
    CaptureOrientation = photoOrientation
};

// Start capture, and pass the context object
AdvancedCapturedPhoto advancedCapturedPhoto = await
    _advancedCapture.CaptureAsync(context);
```

In the **OptionalReferencePhotoCaptured** event handler, cast the **Context** property of the **OptionalReferencePhotoCapturedEventArgs** object to your context object class. This example modifies the file name to distinguish the reference frame image from the final HDR image and then calls the **SaveCapturedFrameAsync** helper method to save the image.

C#

```
private async void
AdvancedCapture_OptionalReferencePhotoCaptured(AdvancedPhotoCapture sender,
OptionalReferencePhotoCapturedEventArgs args)
{
    // Retrieve the context (i.e. what capture does this belong to?)
    var context = args.Context as MyAdvancedCaptureContextObject;

    // Remove "_HDR" from the name of the capture to create the name of the
reference
    var referenceName = context.CaptureFileName.Replace("_HDR", "");

    using (var frame = args.Frame)
    {
        await SaveCapturedFrameAsync(frame, referenceName,
            context.CaptureOrientation);
    }
}
```